

SkyNet Security Camera System

Project outline with a detailed Linux endpoint-agent specification

Prepared from the Teams project chat, “Project notes,” and
“Network_and_Application_Telemetry.pdf.”

Core idea: A local agent observes a Linux computer, normalizes security-relevant telemetry, and sends it to a backend that explains suspicious activity in a web dashboard.

Primary target: Ubuntu/Debian VM. Candidate implementation: Rust or C++.

1. Project scope and responsibilities

The product makes security information scattered across a computer visible in one place. It has three cooperating pieces: an endpoint agent, a backend/data layer, and a dashboard/alert layer.

Team member	Current responsibility	Dependency
Charles Green	Endpoint agent; collect and package telemetry; create the Linux test environment.	Needs the agreed event schema from Josh and Moustafa.
Alasia Powell	Frontend dashboard; initial frontend uploaded.	Needs stable API/event fields.
Jacob Bitter	Backend/API and dashboard integration; README and run instructions.	Must align architecture with the agent.
Josh Radke	Telemetry scenarios, suspicious-event examples, source research, presentation coordination.	Works with Moustafa on collection requirements.
Moustafa Kashen	Telemetry/detection ideas and source cross-referencing; contributed the network/application PDF.	Provides event rationale and detection context.

The chat has some interchangeable assignments. This document treats the agent as Charles’s primary component and defines an explicit contract for Jacob’s backend.

2. Endpoint agent mission

The agent is a small, long-running Linux service that observes security-relevant state and events, converts them to a consistent schema, applies lightweight local validation, buffers them safely, and forwards them to the backend.

Goal	Requirement
Coverage	Application/process, system/file, authentication, network, errors, and external-media telemetry.
Low noise	Prefer event-driven collection; use polling only where necessary and record collection gaps.
Portable	Discover paths and capabilities at runtime; support Ubuntu/Debian and document packages and permissions.
Safe	Do not collect passwords, browser credential contents, file contents, or unnecessary personal data. Redact or hash sensitive values.
Debuggable	Structured logs, health status, configuration validation, clear failures, and a fixture/replay mode.
Backend-friendly	Stable event names, timestamps, host identity, severity, source, and evidence fields.

3. Agent architecture

Recommended pipeline: collectors produce raw observations; the normalizer maps them into canonical events; the context layer enriches them; local rules identify obvious sequences; a queue buffers them; transport sends batches to the backend.

Module	Responsibilities	Failure behavior
Service runner	Start/stop, signal handling, health checks, graceful shutdown.	Exit code and diagnostic log on fatal failure.
Collectors	Read /proc, /sys, auth/system logs, udev/journal, network state, cron/systemd and safe browser metadata.	Raw observation plus source and access status.
Normalizer	Map source-specific records to canonical event types and field names.	Quarantine malformed records; never silently drop them.
Context enricher	Attach host, user, process tree, IP, parent event, baseline status, and confidence.	Unknown context stays null with a reason.
Local preprocessor	Detect failed-login→success, new-user→sudo-change, USB-mount→execution, and similar sequences.	Candidate alert with rule ID and supporting event IDs.
Queue/spool	Bounded memory queue plus disk spool during backend outages.	Backpressure metrics and documented expiry policy.
Transport	HTTPS/JSON or agreed protocol, batching, retry/backoff, authentication, idempotency.	No silent loss; no infinite retry loop.
Configuration	Intervals, enabled collectors, endpoint, credentials/certs, redaction, retention, debug mode.	Validate at startup; never log secrets.

4. Canonical event schema

Every event should carry common metadata, with category-specific fields under *details*.

```
{ event_id, schema_version, event_type, category, timestamp, observed_at, host_id, hostname, actor, severity, source, details, related_event_ids, collector_status }
```

Field	Meaning	Example
event_id	Unique ID for deduplication and correlation.	uuid
event_type	Stable event name.	auth.login_failed
category	auth, process, file, network, system, usb, application, error.	auth
timestamp	Event time; UTC preferred.	2026-10-04T18:30:00Z
host_id	Stable installation/VM identity, not a raw secret.	host-ubuntu-01
actor	User, UID, PID, parent process, or remote peer.	uid=1000
source	Path/tool that produced the observation.	/var/log/auth.log
details	Category-specific fields with redaction.	ip, port, pid, path
related_event_ids	Events supporting a correlation.	[id1,id2]

Use a schema version from the first commit. Agent and backend should reject or quarantine unknown versions rather than guessing.

5. What the agent should collect

This list consolidates the two recently shared documents. The agent records evidence; detection rules decide how concerning the evidence is.

Domain / event	Source	Minimum fields	Suspicious context
Failed login	/var/log/auth.log; bttmp/lastb	User, IP, service, time, count/window	>5 failures from one IP/user in a minute.
Successful login	auth.log; wtmp/last	User, IP, service, TTY, time	Odd hour, new IP, unusual user.
Root/admin login	auth.log	User, method, source, session	Direct root login, especially SSH.
sudo use	auth.log	Actor, command, target, time	First-time admin command.
New user/group change	auth.log; useradd/usermod	Account, actor, group, before/after	New account or normal user added to sudo.
Password change	auth.log	Account, actor, time	Immediately after login failures.
Process start/stop	/proc; audit where available	PID, executable, path, user, parent, time	Unseen program; web server launching shell.
Command line	/proc/<pid>/cmdline	Arguments with redaction, PID, user	Encoded or very long command.
CPU/memory use	/proc	CPU, memory, duration, executable	Unknown process near 100% CPU.
Persistence	cron, systemd, ~/.bashrc, autostart	Path, change, owner, hash, command	New or modified startup item.
File changes	inotify/auditd; selected paths	Path, create/modify/delete, actor, hash/size	Config/log changes or mass changes.
Listening port	/proc/net/tcp; ss -tulpn	Protocol, local addr/port, PID, process	Unexpected/high/random port.
Outbound connection	/proc/net/tcp; ss -ntup	Remote IP/port, process, user, state	Unknown or known-malicious IP.
DNS request	systemd-resolved; syslog	Domain, requester, result, time	Bad/unknown domain or high volume.
Firewall change	ufw log; command audit	Actor, rule before/after, time	Firewall disabled or permissive rule.
ARP/VPN/SSH tunnel	/proc/net/arp/dev; auth.log; lsof	Interface, peer, MAC, tunnel/forward	Duplicate MAC/IP or unexpected tunnel.
USB/device access	udevadm, dmesg, journal, /sys, lsusb	Vendor/product/serial, type, mount, actor	Unknown USB, keyboard-like device, execution from /media or /mnt.
Web/database/app	nginx, MySQL, app logs	Service, request/status, error, client, time	Many 404s or repeated DB auth errors.
Browser extension metadata	Browser profile extension dirs	Browser, extension ID/name, change time	New extension; never collect credential contents.
Mic/camera access	syslog; /dev/video*, /dev/snd	Process, user, device, access time	Background access without expected UI.
Errors/tampering	journalctl, dmesg, log metadata	Service, class, count, log state	Crashes, OOM, denials, logs cleared/stopped.

6. Correlation rules

Rule	Sequence	Evidence
Brute-force then success	Failed logins followed by success from same IP/user within a window.	All login events, count, IP, user, window.
Account persistence	New user followed by sudo/admin-group change.	useradd and group events, before/after.
USB execution	Unknown USB plugged in, mounted, then process starts from mount path.	USB identity, mount event, process path/PID/parent.
Web-to-shell	Web server launches shell/interpreter.	Parent/child process tree and command line.
Download-to-run	Download followed by execution or persistence.	File hash/size, process, startup change.
Log tampering	Log shrinks/clears or logging stops near another event.	File metadata, service state, neighboring events.

Output should include rule_id, severity, confidence, explanation, first_seen, last_seen, and related_event_ids. The dashboard can then explain why an alert exists.

7. Runtime behavior and safety

- **Startup:** validate configuration, discover host identity, check permissions, register collectors, emit agent.started.
- **Runtime:** consume logs/events, sample process/network state only where needed, normalize, enrich, correlate, enqueue, and send.
- **Shutdown:** flush the queue, emit agent.stopped, and exit cleanly.
- **Privileges:** least privilege; use narrowly scoped capabilities or a documented service account.
- **Polling:** begin with 5-15 second sampling where streams are unavailable; make intervals configurable.
- **Resilience:** track rotated log offsets/inodes; use bounded disk spool and exponential backoff.
- **Privacy:** redact passwords, tokens, command-line secrets, browser credential data, and file contents.
- **Tamper resistance:** protect config/spool permissions and report agent stop or config changes after reconnect.

8. Test fixture and backend handoff

Charles should provide a small fixture before the full collector is complete so Jacob can build against stable data.

File	Purpose
events.jsonl	Canonical events covering auth, process, network, USB, application, and error examples.
alerts.json	Expected correlations and explanations for the three explicit scenarios.
host.json	Host metadata and agent version without secrets.
README.md	Ubuntu/Debian setup, packages, permissions, configuration, and fixture replay.

Recommended first milestone: auth login events, process starts, listening/outbound network snapshots, and USB mount/execution correlation. Add remaining collectors after schema and transport are proven.

9. Definition of done

- Runs as a documented Linux service on Ubuntu/Debian VM.
- Emits versioned structured events for the agreed minimum telemetry.
- Records source, timestamp, host, actor/process context, and collection errors.
- Supports brute-force→success, new-account→privilege-change, and unknown-USB→execution.
- Buffers during backend outages and does not silently lose events.
- Uses least privilege and redacts sensitive values.
- Ships with fixtures, tests, configuration examples, and README instructions.
- Backend can consume the fixture and live contract without reverse-engineering the collector.

Source note: This outline is based on the Teams project chat, "Project notes," and the four-page "Network_and_Application_Telemetry.pdf." Details not explicitly decided by the team are presented as recommended design decisions.